

Housing for international students

Build spec for the Product Space UIUC Fall Product Hackathon, Oct 2 to 3, 2026. Submission is due Saturday at 5:15pm.

1. The problem

Most landlords near campus ask for a Social Security number, US credit history, or a US cosigner. A lot of international students have none of these. Some buildings will accept a passport, a guarantor service, or prepaid rent instead, but listing sites don't say which ones. Students find out by applying and getting rejected. Then, once they do find a place, they sign a US lease without knowing which clauses are normal and which will cost them money.

Our app fixes two steps where these students get stuck:

- **Finding a building that will accept them.** Students enter what documents they have, and the app shows only the buildings they can actually get, with the reason for each.
- **Understanding the lease.** The app pulls out the clauses that matter (deposit, fees, early termination and so on) and explains each one in plain language, next to the original text.

Roommate matching and building groups are extras. We show them as clickable screens with sample data, not working features.

2. What we build this weekend

Feature	Demo build	Why
Qualification filter	Working code	Core of the pitch. Small and testable.
Lease walkthrough	Working code	The part judges will remember. Also the riskiest, so start it first.
Roommate matching	Clickable screens with sample data	Nice extra. The scoring formula is in section 6 if someone has spare time.
Building groups	Clickable screens with sample data	Needs real logins to mean anything.
Landlord dashboard	Skip, or one static screen	We type building data in by hand.
Logins	Skip. Pretend the user is signed in.	Not what we're judged on.

3. How the pieces fit

Think of the app as a restaurant. Everyone on the team should be able to point at which part of the restaurant their code lives in.

Part	Restaurant	What it does here
Frontend	Dining room	Forms, building list, lease viewer. Runs in the browser.

Part	Restaurant	What it does here
Backend	Kitchen	Runs the filter, calls the AI, decides what data each user may see.
Database	Pantry	Students, buildings, leases, questionnaire answers.
AI service (Claude API)	Specialist chef	Classifies and explains lease clauses. Nothing else.

Two rules follow from this picture:

- **The filter and roommate scoring are plain code, not AI.** The same input must always give the same answer, and we must be able to say why. AI only handles the lease, because reading legal language is the one job fixed rules can't do.
- **The Claude API key lives on the backend only.** Anything in frontend code is visible to anyone who opens browser dev tools. If the key ships to the browser, anyone can copy it and spend our credits. The frontend calls our backend route, and the backend calls Claude.

Suggested stack. Next.js for frontend and backend routes, Supabase for the database and file storage, Claude API for the lease step. If the team already knows something else well, use that instead. Learning a framework mid-hackathon costs hours we don't have.

4. Data model

Four tables. A lease points to a student and a building, and a group membership points to a student and a building.

Students

Field	Type	Notes
id, name	id, text	
has_ssn, has_us_cosigner, has_us_credit	yes / no	
can_prepay	yes / no	Can they pay several months up front?
visa_type	text	Display only for the demo.
budget_monthly	number	Dollars per month.
move_in_date	date	
language	text	Language for lease explanations.
roommate_answers	JSON	See section 6.

Buildings

Field	Type	Notes
id, name, address		
rent_min, rent_max	number	Cheapest and most expensive unit.
requires_ssn	yes / no / unknown	

Field	Type	Notes
accepts_passport_instead_of_ssn	yes / no / unknown	
requires_us_cosigner	yes / no / unknown	
accepts_guarantor_service	yes / no / unknown	e.g. TheGuarantors, Leap
allows_prepayment, prepay_months	yes / no / unknown, number	
lease_type	joint / individual	
remote_tour, e_signing	yes / no / unknown	
last_confirmed	date	When our team last checked with the building.

Design decision: three states, not two. Every requirement field is yes, no, or unknown. With plain true/false, a building we never asked about looks the same as a building that said no. Then the filter either hides a building that would have accepted the student, or shows one that will reject them. Unknown keeps that difference visible. When someone types in building data and doesn't know an answer, they leave it unknown, never guess.

Leases and group memberships

Table	Fields
leases	id, student_id, building_id, pdf_path, clauses (list of: clause_id, heading, original_text, category, explanation)
group_memberships	student_id, building_id, verified (yes / no), visible (yes / no)

5. Feature: qualification filter

Input. The student's form answers: documents, can_prepay, budget, move-in date.

Output. Two lists of buildings: *You qualify* and *Needs confirming* (we're missing data). Buildings that fail a rule are hidden. Every building in the first two lists carries the reasons that put it there.

Each rule has an "unless" built in. That's the whole product. A normal listing site knows "needs SSN." We also know "but will accept a passport instead."

```

check_building(student, building):
    reasons = []      # why they qualify, shown as labels
    unknowns = []     # what we couldn't confirm

    # SSN rule
    if building.requires_ssn == yes and not student.has_ssn:
        if building.accepts_passport_instead_of_ssn == yes:
            reasons.add("Accepts passport instead of SSN")
        elif building.accepts_passport_instead_of_ssn == unknown:
            unknowns.add("Passport instead of SSN?")
        else:
            return FAIL
    if building.requires_ssn == unknown and not student.has_ssn:
        unknowns.add("SSN required?")

    # Cosigner rule: two possible ways around it
    if building.requires_us_cosigner == yes and not student.has_us_cosigner:
        if building.accepts_guarantor_service == yes:
            reasons.add("Accepts guarantor service")
        elif building.allows_prepayment == yes and student.can_prepay:
            reasons.add("Accepts " + building.prepay_months + " months prepaid")
        elif either of those is unknown:
            unknowns.add("Guarantor or prepayment?")
        else:
            return FAIL
    if building.requires_us_cosigner == unknown and not student.has_us_cosigner:
        unknowns.add("Cosigner required?")

    # Budget: compare against the CHEAPEST unit
    if building.rent_min > student.budget_monthly:
        return FAIL
    if building.rent_max > student.budget_monthly:
        reasons.add("Some units are over your budget")

    if unknowns is empty: return QUALIFY with reasons
    else:                 return NEEDS_CONFIRMING with reasons, unknowns

```

Why it's shaped this way:

- **The filter returns reasons, not just pass or fail.** The labels the student sees ("Accepts guarantor service") come straight from the rule that passed. If the labels were written somewhere else, they could drift out of sync with the real logic and tell the student something the filter never checked.
- **Budget uses rent_min.** A building has a range. Comparing the budget to rent_max would hide buildings with an affordable unit. Comparing it to rent_min and labeling the rest is honest about both.
- **Prepayment is a way around the cosigner rule,** so it only helps if the student can actually prepay. That's why the form asks.

Data for the demo. Our team calls about ten buildings and types in their answers. Fill in last_confirmed with the call date. Show it on each building card so students can see how fresh the information is.

Done when the filter has small tests for at least these cases: student with no SSN and a passport-accepting building (qualifies), the same student and a building that says no (hidden), a building with unknown fields (needs confirming), and a building whose cheapest unit is over budget (hidden).

6. Feature: lease walkthrough

This is the most technical piece and the biggest demo risk. Whoever owns it should get one lease working end to end in the first few hours, even if ugly.

Step	What happens	Demo shortcut
1. Upload	Student uploads a lease PDF. Save it to file storage, link it to student and building.	
2. Extract text	Pull the text out of the PDF.	Text PDFs only. If no text comes out, it's a scan. Show "This looks like a scanned document. Try a digital copy." OCR is out of scope.
3. Split into clauses	Cut the text at numbered sections or headings. Give each a clause_id.	Use 2 or 3 prepared leases with clean numbered sections.
4. Classify	Ask the AI which clauses are about deposit, liability, early termination, subletting, renewal, fees, or none.	One call per lease. Send all clauses with their ids, get back JSON.
5. Explain	For each flagged clause, ask the AI to explain it in plain language in the student's language.	
6. Show side by side	Explanation on one side, original clause on the other.	

Why clause by clause, instead of asking the AI to summarize the whole lease:

A summary of a 20-page lease can quietly drop or invent details, and the student can't check it. Working clause by clause ties every explanation to text the student can read for themselves. If the AI gets something wrong, the error sits right next to the real wording. For a document with real money attached, that's the feature, not decoration. When a judge asks "what if the AI is wrong?", this is our answer.

Rules that keep the side-by-side honest. (1) The original text on screen always comes from our own split in step 3, never from the AI's output. If we display text the AI returned, it can be paraphrased or made up and still look like a quote. (2) When the classify step returns JSON, throw away any clause_id that doesn't exist in our split. (3) If the JSON fails to parse, show the lease with no flags and an error message rather than crash the demo.

Why one classify call per lease, not one per clause. A lease can have 40 sections. Forty calls in a row is slow on stage. One call with every clause labeled by id is faster, and traceability survives because the ids map back to our text. Explanations (step 5) stay one call per flagged clause, since that's usually 5 to 8 calls and each explanation should focus on one clause.

Every lease page shows: "This is not legal advice" and a link to the UIUC Tenant Union for a real review.

Privacy for the demo. Use sample leases or ones with names and addresses blacked out. Don't upload a friend's real lease to an API without asking them.

7. Mock: roommate matching

Screens only for the demo. If someone wants to build it, here is the formula so the screens and any code agree.

Questionnaire. Sleep schedule, cleanliness, guests, noise, cooking, each on a 1 to 5 scale. For each, the student marks importance (low = 1, medium = 2, high = 3) and can mark it a dealbreaker with a limit.

Step 1, filter. Remove anyone who breaks a dealbreaker. Check both directions. If Alex says "no smokers," Sam can't smoke, *and* if Sam says "no night owls," Alex can't be one. A one-way check would match Alex with someone who has already ruled Alex out.

Step 2, score.

```
closeness(a, b)    = 1 - |a - b| / 4          # 1.0 = same answer, 0.0 = opposite
score_for(A, B)    = sum(A.importance[q] * closeness(A[q], B[q]))
                   / sum(A.importance[q]) * 100
match(A, B)        = (score_for(A, B) + score_for(B, A)) / 2
```

The last line matters. Alex and Sam weigh questions differently, so Alex's view of the match and Sam's view give different numbers. Averaging them means the match score is the same no matter whose screen it appears on.

Filter first, then score, is one of the most common shapes in software. Job sites, dating apps and search engines all do a hard cut and then rank what's left.

8. Mock: building groups

Screens only. In a real version, the backend checks two things before showing anyone: does the viewer have a signed lease at this building, and did each person shown opt in? That check must live in the backend. Frontend code runs in the user's browser and anyone can edit it. The kitchen decides what leaves the kitchen.

Show name, photo, and optionally hometown or interests. Never floor or unit number.

9. Who owns what

A suggested split for four people. Each person owns one piece end to end, so at any hour it's clear who to ask.

Role	Owns	Owner
Lease pipeline	Steps 1 to 6 of section 6, Claude prompts, JSON checks	
Filter and data	Database tables, filter logic and its tests, the backend route the form calls	
Frontend	Student form, results list, side-by-side lease viewer, mock roommate and group screens	
Research, testing, pitch	Call the ten buildings, run user tests, build the demo script and slides	

Build order. First get the lease pipeline working end to end on one lease and the filter working on hard-coded data. Connect them to the UI after that. Polish and mocks come last. If something has to be

cut at 2pm Saturday, cut from the bottom of this list.

10. How we prove it works

The prompt asks who can now succeed who couldn't before, so the demo should show a number, not just a feature tour.

- **Filter test.** Give an international friend our ten buildings on a normal listing site and ask which ones they could rent. Then ask with our app. Count correct answers both ways.
- **Lease test.** Give 3 to 5 friends a sample lease and ask a concrete question, like "How much do you pay if you leave three months early?" Time them and check the answer, once with just the PDF and once with our walkthrough.

Even three observed tests give us a real before-and-after slide. Most teams won't have one.

11. If this goes to real users

Everything above is a demo. Before real students rely on it, these stop being optional and need a much higher bar:

- Real login and verification, because groups depend on knowing who lives where.
- Secure storage for leases and documents. They hold names, addresses and financial details.
- Building data that gets re-confirmed on a schedule. A stale "accepts passports" sends a student to apply somewhere that will reject them, and an application fee is real money.
- OCR for scanned leases, since many landlords send scans.
- The side-by-side clause view stays no matter what. It's the main protection against the AI being wrong.

12. Decide at kickoff

- Stack: the suggestion above, or what the team already knows?
- Which ten buildings, and who calls them?
- Which languages does the lease explanation support in the demo? Pick one or two we can check ourselves.
- Which sample leases do we use, and are personal details removed?